

hw1-written
Haohai Pang
Feb 3, 2024

Problem 1b:

input_ids: it serves as a mapping (token -> int) for tokens, and each token has a unique id. The text's input_ids will be an ordered list corresponding to the order of tokens in the original text.

token_type_ids: it tells which segment/sequence each token belongs to. Tokens from the first sequence will be 0s, those from the second sequence will be 1s. It gives a list of 1/0 of the same order of tokens in the original text. Particularly, padding tokens will be assigned to 0.

Attention_masks: it tells which token the model should pay attention. In BERT, it is a binary variable where padding tokens are considered less useful (assigning 0s), where other tokens from either the first or second sequence will be assigned to 1s, following the same order.

The length of the three lists is determined by 'max_length', which is a hyperparameter.

Problem 1c:

There are two hyperparameters that are tuned for each task: batch size and learning rate. Each has several options: **batch sizes**= [8, 16, 32, 64, 128], **learning rates** = [3e-4, 1e-4, 5e-5, 3e-5]. Epochs = 4 is fixed.

Second, we work on every **evaluation metrics**. Since GLUE containing multiple scores, we want to figure out which hyperparameters work best for each task. So even for the same model, we wish to find the best configuration under specific tasks.

Next, we **cross validate** (using GridSampler) each combination of hyperparameters to determine which one yields the best result. This combination would be the finetuned hyperparameters. Of course, they should be tested under the exact same data and cross-validation strategy. Finally, the fixed hyperparameters will be used for the test set and be reported under the table.

Problem 3a:

	Validation Accuracy	Learning Rate	Batch Size
Without Bitfit	0.8892	3e-4	128
With Bitfit	0.6488	3e-4	8

Problem 3b:

	# Trainable Parameters	Test Accuracy
Without Bitfit	4386178	0.87472
With Bitfit	3074	0.6458424925804138

The Bitfit method does not perform as good as described in the original paper. The validation accuracy and test accuracy drop significantly with Bitfit. This is expected as the number of trainable parameters is much less, so there is less “freedom” for parameters to be optimized. From the empirical results, it is not true that Bitfit works better than fine-tuning the entire model. I will say Bitfit is still promising as the time and recourse needed for training becomes shorter. But if you are most interested in getting a better result (e.g., classification accuracy), it is desirable to finetune the entire model.